

Induction Proof

Definition

Induction proof is a powerful technique for proving statements about all natural numbers (0, 1, 2, 3,...). It relies on establishing two key steps: a *base case* and a *recursive step*. The base case demonstrates that the statement holds true for the starting value of the sequence (typically 0 or 1). The recursive step shows how, if the statement is true for some number n , it must also be true for the next number, $n+1$. This creates a chain reaction – if both steps are proven, we've demonstrated that the statement holds for all numbers in the sequence. We often refer to the assumption made in the recursive step as the *induction hypothesis*.

Picture this

Imagine you're building with LEGO bricks. The base case is like showing you have one brick—it's a valid configuration. The recursive step is demonstrating that if you already have 'n' bricks arranged in a specific way, you can always add another brick to extend the structure according to the rules of your building set. You don't need to show *every* possible arrangement; just showing how adding one more brick maintains the correct configuration is sufficient.

Worked example

Let's prove that the sum of the first n natural numbers is given by the formula:

$$S_n = \frac{n(n+1)}{2}.$$

Base Case (n=1): When $n=1$, $S_1 = 1$. The formula gives us $\frac{1(1+1)}{2} = \frac{2}{2} = 1$. The formula holds true for the base case.

Recursive Step: Assume that $S_k = \frac{k(k+1)}{2}$ is true for some integer $k \geq 1$. We want to show that $S_{k+1} = \frac{(k+1)(k+2)}{2}$. We know that $S_{k+1} = S_k + (k+1)$. Substituting our assumption, we get:

$$S_{k+1} = \frac{k(k+1)}{2} + (k+1) = \frac{k(k+1)+2(k+1)}{2} = \frac{(k+1)(k+2)}{2}.$$

This confirms that if the formula holds for k , it also holds for $k+1$.

Problem-solving application

Consider a recursive algorithm to calculate the factorial of a non-negative integer n . A naive implementation might have errors, particularly with small values of n . Induction proof can be used to verify that such an algorithm correctly computes factorials. For example: if your algorithm attempts to compute $n!$ and it has an error in the base case (i.e., doesn't correctly handle $n=0$ or $n=1$), then the entire result will be incorrect for all values of n . The core idea is that by establishing a correct base case, we build up our proof from the ground up, ensuring correctness at every step of the recursion.